

Vermont Health Platform — Complete Setup Guide

This guide walks you through every external service, credential, and configuration step required to run the application. Follow the sections in order. Do not skip ahead.

Table of Contents

- 1. [Prerequisites](#)
 - 2. [Clone & Install](#)
 - 3. [Supabase — Database & Auth](#)
 - 4. [Sanity CMS](#)
 - 5. [Google Gemini — AI Backend](#)
 - 6. [Python AI Backend](#)
 - 7. [Stripe — Payments](#)
 - 8. [Frontend Environment Variables](#)
 - 9. [Running the App Locally](#)
 - 10. [Creating Your First Admin User](#)
 - 11. [Ongoing Maintenance — Critical Warnings](#)
 - 12. [Production Deployment](#)
-

1. Prerequisites

Install the following before starting:

- **Node.js** v20+ — <https://nodejs.org>
- **Python** 3.11+ — <https://python.org>
- **Git** — <https://git-scm.com>

Verify:

```
node --version    # should print v20.x or higher
python3 --version # should print 3.11.x or higher
```

2. Clone & Install

```
git clone <your-repo-url>
cd Vermont-Health-Platform

# Install frontend dependencies
cd frontend
npm install
cd ..

# Install Python backend dependencies
cd backend
python3 -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt
cd ..
```

3. Supabase — Database & Auth

Supabase is the database and authentication system. Every user account, role, and subscription record lives here. **This must be set up before the app will work at all.**

3.1 Create a Supabase Project

1. Go to <https://supabase.com> and sign in (or create a free account)
2. Click **New Project**
3. Fill in:
 - **Name:** Vermont Health Platform (or anything you like)
 - **Database Password:** Choose a strong password — **save it somewhere safe**
 - **Region:** Choose the closest to you (e.g. US East)
4. Click **Create new project**
5. Wait 1–2 minutes for the project to finish provisioning

3.2 Get Your Credentials

In the Supabase dashboard, go to **Settings → API**. Copy:

Variable	Where to find it
NEXT_PUBLIC_SUPABASE_URL	"Project URL" field
NEXT_PUBLIC_SUPABASE_ANON_KEY	"anon public" key
SUPABASE_SERVICE_ROLE_KEY	"service_role secret" key — keep this private

Also go to **Settings → API → JWT Settings** and copy:

Variable	Where to find it
SUPABASE_JWT_SECRET	"JWT Secret" field

3.3 Run the Database Migrations

The app requires specific database tables. You must run the migration files in order.

In the Supabase dashboard, go to **SQL Editor** and run each file below one at a time. Copy the full contents of each file, paste into SQL Editor, click **Run**, wait for "Success", then move to the next.

Run in this exact order:

1. [supabase/migrations/001_profiles_and_roles.sql](#)
2. [supabase/migrations/002_content_data.sql](#)
3. [supabase/migrations/003_academy.sql](#)
4. [supabase/migrations/004_advisory.sql](#)
5. [005_rag_vectors.sql](#) — **SKIP THIS** (requires pgvector, set up separately later)
6. [supabase/migrations/006_rls_policies.sql](#)

Each migration should complete with no errors. If you get an error, stop and fix it before running the next one.

3.4 Configure Auth Settings

In the Supabase dashboard, go to **Authentication → URL Configuration**:

- **Site URL:** <http://localhost:3000> (for local dev)
- **Redirect URLs:** Add <http://localhost:3000/auth/callback>

For production, you will add your Vercel domain here as well.

3.5 ⚠ CRITICAL — Supabase Free Tier Pausing

The Supabase free tier pauses your project after 1 week of inactivity. When paused, every database call fails silently with "Failed to fetch" — the app appears broken for no obvious reason.

- What to watch for:** Any "Failed to fetch" error that wasn't there before.
- How to fix:** Go to <https://supabase.com/dashboard> → find your project → click **Restore project** → wait 2 minutes → hard-refresh your browser.
- How to prevent:** Log into Supabase at least once a week, or upgrade to Pro (\$25/mo) which disables auto-pausing.

4. Sanity CMS

Sanity is the content management system for articles, academy modules, glossary terms, and all editorial content.

4.1 Get Your Credentials

You already have a Sanity project (`fxz10xl7`). Get your credentials from <https://sanity.io/manage>:

1. Select your project
2. Go to **API → Tokens**
3. Click **Add API token**
 - Name: `HTR Backend`
 - Permissions: **Editor**
4. Copy the token immediately — it is only shown once

Variable	Value
<code>NEXT_PUBLIC_SANITY_PROJECT_ID</code>	<code>fxz10xl7</code>
<code>NEXT_PUBLIC_SANITY_DATASET</code>	<code>production</code>
<code>SANITY_API_TOKEN</code>	The token you just created

4.2 Add Content

The app will work without Sanity content — it falls back to static data — but pages like the glossary and academy modules will be empty.

To add content, run the Sanity Studio:

```
# If you have a studio directory:
cd studio
npm install
npm run dev
# Open http://localhost:3333
```

5. Google Gemini — AI Backend

The AI Analyst chat feature uses Google's Gemini model. Without this, the chat page will not work.

5.1 Get a Gemini API Key

1. Go to <https://aistudio.google.com/app/apikey>
2. Click **Create API key**
3. Copy the key

Variable	Where used
<code>GOOGLE_API_KEY</code>	<code>backend/.env</code> — used by the Python AI backend

5.2 Check Your Quota

The free Gemini tier has rate limits. If you see errors in the AI chat after heavy use, you may have hit your quota. Check at: <https://aistudio.google.com/app/apikey> → "Check quota"

6. Python AI Backend

The AI backend is a separate Python service that must be running alongside the Next.js frontend for the AI Analyst chat to work.

6.1 Create the Backend Environment File

```
cd backend
cp .env.example .env
```

Open `backend/.env` and fill in:

```
GOOGLE_API_KEY=your_gemini_api_key_here

SANITY_PROJECT_ID=fxz10xl7
SANITY_DATASET=production
SANITY_API_TOKEN=your_sanity_token_here
SANITY_API_VERSION=2023-10-01

SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key_here
SUPABASE_JWT_SECRET=your_jwt_secret_here

FRONTEND_URL=http://localhost:3000
```

6.2 Start the Backend

```
cd backend
source venv/bin/activate # Windows: venv\Scripts\activate
uvicorn main:app --reload --port 8000
```

First run is slow (2–5 minutes) — it indexes all content from Sanity and any PDFs in `backend/data/`. Subsequent runs are fast (~2 seconds) because the index is cached.

You will see this when it is ready:

```
INFO:      Application startup complete.
```

Verify it is running: open `http://localhost:8000/health` in your browser. You should see `{"ok": true, "index_ready": true}`.

6.3 Add PDF Content (Optional)

Drop any PDF files into `backend/data/` before starting the backend. They will be indexed automatically on startup and become part of the AI's knowledge base.

6.4 ⚠ The Backend Must Always Be Running

The frontend proxies all AI chat requests to `http://localhost:8000`. If the Python backend is not running, the chat page will show an error. You need **two terminals open** whenever developing:

- Terminal A: `cd backend && uvicorn main:app --reload --port 8000`

- Terminal B: `cd frontend && npm run dev`

7. Stripe — Payments

Stripe handles subscriptions and billing. Without this, the pricing page and upgrade flow will not work. The rest of the app functions fine without Stripe configured.

7.1 Create a Stripe Account

1. Go to <https://stripe.com> and create an account
2. Stay in **Test mode** (the toggle in the top left) for development
3. Never use live mode keys in development

7.2 Get Your API Keys

In the Stripe dashboard, go to **Developers → API keys**:

Variable	Value
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>	Publishable key (starts with <code>pk_test_</code>)
<code>STRIPE_SECRET_KEY</code>	Secret key (starts with <code>sk_test_</code>) — keep private

7.3 Create Your Products and Prices

In Stripe dashboard → **Product catalog → Add product**:

Create a product called **"HTR Subscriber"** with a recurring monthly price (e.g. \$19/mo). After saving, copy the **Price ID** (starts with `price_`).

Variable	Value
<code>STRIPE_SUBSCRIBER_PRICE_ID</code>	The price ID you just copied

7.4 Configure the Webhook

The webhook keeps your database in sync when a subscription is created, updated, or cancelled. Without it, user roles will not update after payment.

For local development, use the Stripe CLI:

```
# Install Stripe CLI: https://stripe.com/docs/stripe-cli
stripe login
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

The CLI will print a webhook signing secret (`whsec_...`). Copy it.

Variable	Value
<code>STRIPE_WEBHOOK_SECRET</code>	The <code>whsec_...</code> secret from the CLI output

For production, create the webhook in the Stripe dashboard:

- **Endpoint URL:** `https://your-domain.vercel.app/api/stripe/webhook`
- **Events to listen for:** `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_succeeded`, `invoice.payment_failed`

8. Frontend Environment Variables

Create the file `frontend/.env.local` with all variables combined:

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key_here
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key_here

# Sanity
NEXT_PUBLIC_SANITY_PROJECT_ID=fxz10xl7
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01
SANITY_API_TOKEN=your_sanity_token_here

# Stripe (optional – only needed for payments)
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_your_key_here
STRIPE_SECRET_KEY=sk_test_your_key_here
STRIPE_WEBHOOK_SECRET=whsec_your_secret_here
STRIPE_SUBSCRIBER_PRICE_ID=price_your_price_id_here

# Python AI Backend
PYTHON_BACKEND_URL=http://localhost:8000

# App URL
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Never commit **.env.local** to git. It is already in **.gitignore**.

9. Running the App Locally

You need two terminals running simultaneously:

Terminal 1 — AI Backend:

```
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000
```

Terminal 2 — Frontend:

```
cd frontend
npm run dev
```

Open <http://localhost:3000> in your browser.

10. Creating Your First Admin User

After setup, create your account and grant yourself admin access.

10.1 Create the Account

1. Go to <http://localhost:3000/signup>
2. Sign up with your email and password

10.2 Verify the Account Exists

In Supabase dashboard → **Authentication** → **Users** — you should see your user listed.

10.3 Grant Admin Access

In Supabase dashboard → **SQL Editor**, run:

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('your-user-uuid-here', 'admin');
```

Replace `your-user-uuid-here` with your UUID from the Authentication → Users page.

To also give yourself subscriber access (required to use the dashboard and AI chat):

```
INSERT INTO public.user_roles (user_id, role)
VALUES ('your-user-uuid-here', 'subscriber');
```

10.4 Available Roles

Role	Access
free	Default for all new users — public pages only
subscriber	Dashboard, AI Analyst chat, HTI tools
student	Academy courses
professional	Professional-tier content
advisory	Advisory hub and reports
admin	Everything

A user can hold multiple roles simultaneously.

11. Ongoing Maintenance — Critical Warnings

These are the things most likely to break the app without an obvious error message.

⚠ Supabase Auto-Pause (Free Tier)

Risk level: HIGH. The app will fail silently with "Failed to fetch" errors.

- **Trigger:** Project is paused after ~1 week of no activity
- **Symptom:** Login fails, database queries return nothing, "Failed to fetch" in console
- **Fix:** Supabase dashboard → click Restore → wait 2 minutes → hard refresh browser
- **Prevention:** Visit Supabase dashboard weekly, or upgrade to Pro

⚠ Python Backend Not Running

Risk level: HIGH. The AI chat will show an error.

- **Symptom:** AI Analyst chat shows "AI Offline" or returns an error
- **Fix:** Open a terminal, `cd backend && uvicorn main:app --reload --port 8000`
- **Check:** `http://localhost:8000/health` should return `{"ok": true}`

⚠ Google Gemini Quota

Risk level: MEDIUM.

- **Symptom:** AI chat works sometimes but returns errors under heavy use
- **Fix:** Check quota at `https://aistudio.google.com` → consider upgrading to paid tier

⚠ Stripe Webhook Not Running (Local Dev)

Risk level: MEDIUM. Payments will process but user roles won't update automatically.

- **Symptom:** User pays but still sees "upgrade" prompt

- **Fix:** Run `stripe listen --forward-to localhost:3000/api/stripe/webhook` in a terminal

⚠ next.config.ts Changes Require Full Server Restart

Risk level: LOW but confusing.

- Any change to `frontend/next.config.ts` (security headers, redirects, etc.) requires stopping the dev server (Ctrl+C) and restarting (`npm run dev`)
- Hot reload does NOT apply to this file

⚠ Sanity Project Token Expiry

Risk level: LOW.

- Sanity API tokens do not expire by default, but if you rotate them, update `SANITY_API_TOKEN` in both `frontend/.env.local` and `backend/.env`

12. Production Deployment

12.1 Frontend — Vercel

1. Push your repo to GitHub
2. Go to `https://vercel.com` → Import project
3. Set **Root Directory** to `frontend`
4. Add all environment variables from Section 8 (use production values, not test keys)
5. Add your Vercel domain to Supabase → Authentication → URL Configuration

12.2 Python Backend — Railway

1. Go to `https://railway.app` → New Project → Deploy from GitHub
2. Select the repo, set root to `backend`
3. Add environment variables from Section 6.1
4. Railway will use the `Procfile` automatically:

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

5. Copy the Railway public URL (e.g. `https://your-backend.railway.app`)
6. Set `PYTHON_BACKEND_URL=https://your-backend.railway.app` in Vercel env vars

12.3 Update Supabase for Production

In Supabase → Authentication → URL Configuration:

- Add your Vercel domain to **Redirect URLs:** `https://your-domain.vercel.app/auth/callback`
- Update **Site URL** to `https://your-domain.vercel.app`

12.4 Update Stripe for Production

- Create a new webhook endpoint in Stripe dashboard pointing to your Vercel domain
- Replace `STRIPE_WEBHOOK_SECRET` in Vercel with the new production webhook secret
- Switch from test mode keys (`pk_test_`, `sk_test_`) to live mode keys (`pk_live_`, `sk_live_`)

Quick Reference — What Controls What

Feature	Service	Config file
User accounts & login	Supabase Auth	<code>frontend/.env.local</code>
Database (roles, subscriptions)	Supabase Postgres	<code>frontend/.env.local</code>

Feature	Service	Config file
Editorial content (articles, modules)	Sanity CMS	<code>frontend/.env.local</code>
AI Analyst chat	Python backend + Google Gemini	<code>backend/.env</code>
Payments & subscriptions	Stripe	<code>frontend/.env.local</code>
App hosting (frontend)	Vercel	Vercel dashboard
AI backend hosting	Railway	Railway dashboard

Last updated: 2026-03-19